



INSTITUTO FEDERAL
Paraíba
Campus Campina Grande

Disciplina: Projeto em Engenharia de Computação I

Período: 2026.1

Equipe: Cefras José Ferreira Mandú De Almeida

Lavoisier Chaves Ramos

Osmar Dos Santos Filho

Orientador(a): Dr. Moacy Pereira da Silva

Visão Computacional para detecção de flechas no alvo e pontuação em tempo real, para modalidade de tiro com arco.

1. Introdução e Contextualização Estratégica

No atual panorama tecnológico, a Visão Computacional consolidou-se como o motor de inovação para sistemas autônomos e analíticos. A convergência entre a linguagem Python e a biblioteca OpenCV permite a transposição de dados multidimensionais brutos — matrizes de intensidade luminosa — em informação estruturada e acionável. Como Arquitetos de Soluções, compreendemos que a eficácia de um sistema não reside apenas na execução do código, mas na robustez do planejamento estratégico que precede a implementação técnica. A transição do "sentido da visão" para as máquinas altera radicalmente a utilidade dos sistemas artificiais. Ao dotar um algoritmo da capacidade de segmentar e reconhecer padrões, transcendemos a mera captura de imagem para alcançar a interpretação semântica. Este relatório estabelece o rigor metodológico necessário para garantir que a arquitetura proposta seja escalável e performante, conectando a necessidade de negócio à viabilidade técnica de processamento de imagem.

2. Consolidação da Proposta: Problema e Objetivos

O desafio técnico central deste projeto consiste na extração de atributos estruturados a partir de matrizes de dados espaciais não estruturados. A presença de ruído eletrônico, variações de iluminância e fundos complexos exige uma abordagem sistemática para que o sistema consiga isolar o objeto de interesse sem perda de integridade geométrica.

Objetivos do Projeto

- **Desenvolver** um pipeline de pré-processamento adaptativo para mitigação de ruído, garantindo a estabilidade dos módulos de segmentação subsequentes.
- **Implementar** binarização baseada no método de Otsu para isolar objetos (255) de fundos (0), cumprindo o requisito lógico de algoritmos de extração de contornos.
- **Extrair** geometrias complexas e relações de vizinhança, utilizando hierarquias para distinguir entre bordas externas e cavidades internas.
- **Otimizar** a representação vetorial dos objetos através de aproximações de cadeias de pontos, reduzindo o overhead de memória RAM.
- **Categorizar** as formas detetadas para integração futura com modelos de aprendizagem supervisionada ou agrupamento estatístico. A coerência entre a definição do problema e estes objetivos é o fator determinante para o sucesso da engenharia. Um desalinhamento nesta fase resultaria em desperdício de ciclos de CPU em dados irrelevantes, comprometendo a latência e a precisão da solução final.

3. Fundamentação Teórica e Estado da Arte (Grounding)

A manipulação de imagens no OpenCV assenta na lógica de matrizes NumPy de tipo uint8 (inteiros sem sinal de 8 bits). Esta representação é fundamental para a eficiência de memória, limitando os valores de intensidade entre 0 (ausência de luz) e 255 (saturação máxima).

Representação e Sistemas de Cores

Um ponto crítico de engenharia é a distinção entre espaços de cor. Enquanto o padrão industrial é o RGB, o OpenCV processa nativamente em **BGR**. Ignorar esta inversão ao utilizar bibliotecas de visualização como a Matplotlib (que opera em RGB) resulta em inversões cromáticas — um erro comum que esta arquitetura mitiga através de conversões explícitas no módulo de aquisição. Para a deteção de formas, a simplificação para binarização é essencial, pois transforma a complexidade tridimensional das cores numa topologia binária que acelera drasticamente a análise geométrica.

Técnicas de Pré-processamento

A qualidade da extração de informação depende diretamente da técnica de suavização aplicada para controlar o ruído:

Técnica de Processamento	Impacto na Qualidade da Extração de Informação
Filtro de Média (Blur)	Reduz ruído de alta frequência, mas causa desfoque acentuado em bordas críticas.
Gaussian Blur	Utiliza um kernel gaussiano para uma suavização mais natural; ideal para ruído de distribuição normal.

Median Blur	Substitui o pixel pela mediana da vizinhança; é a técnica ideal para remover ruído "sal e pimenta" sem criar novas cores.
Filtro Bilateral	Opera no domínio do espaço e da intensidade; remove ruído preservando rigorosamente a nitidez das bordas.

A escolha de imagens binarizadas para a detecção de contornos não é apenas uma otimização de velocidade; é uma exigência algorítmica. Funções como `findContours` assumem que o objeto de interesse é branco (255) sobre um fundo preto (0), permitindo a navegação eficiente pela matriz.

4. Modelagem Conceitual da Solução

A arquitetura lógica do sistema é estruturada em cinco subsistemas principais:

1. **Módulo de Aquisição:** Interface com sensores para captura e carregamento via `imread`, tratando a origem das coordenadas no canto superior esquerdo (0,0).
2. **Módulo de Pré-processamento:** Normalização de histograma e suavização seletiva (Gaussian/Bilateral) para limpeza da matriz.
3. **Módulo de Segmentação:** Conversão via `cvtColor` (BGR to Gray) e binarização adaptativa para isolamento de regiões de interesse.
4. **Módulo de Extração de Contornos:** Execução da função `findContours` para reconhecimento geométrico.
5. **Módulo de Classificação Semântica:** Atribuição de significado aos dados extraídos (ex: K-Means ou CNN).

Hierarquia e Eficiência de Dados

Para o mapeamento estrutural, utilizaremos o modo de recuperação `RETR_TREE`. Esta flag permite reconstruir a árvore hierárquica completa (Pai-Filho-Neto), fundamental para distinguir, por exemplo, a moldura de um objeto dos seus furos internos. Caso a aplicação exigisse apenas o contorno externo, o modo `RETR_EXTERNAL` seria preferível para poupança de recursos. No que toca à representação de pontos, a solução adotará o método `chain_approx_simple`. Em vez de armazenar todos os pixels da borda (que consumiria memória linearmente com o perímetro), este método armazena apenas os pontos de inflexão (como os quatro cantos de um retângulo). Este *trade-off* garante alta precisão geométrica com um consumo de memória drasticamente reduzido, permitindo a escalabilidade para processamento de vídeo em tempo real.

5. Definição de Requisitos do Sistema

O rigor na definição de requisitos assegura a confiabilidade da solução em ambientes críticos, como linhas de montagem automatizadas ou diagnóstico médico.

Requisitos Funcionais

- O sistema deve converter obrigatoriamente a imagem de BGR para escala de cinza antes da segmentação.
 - O algoritmo deve respeitar o sistema de coordenadas com origem (0,0) no topo superior esquerdo da matriz.
 - A solução deve suportar a extração de hierarquias completas para identificação de cavidades internas nos objetos.
 - O sistema deve permitir a sobreposição visual (overlay) dos contornos detetados na imagem original para auditoria.
- Requisitos Não Funcionais**
- **Desempenho:** O pipeline completo de processamento por frame não deve exceder os 100ms.
 - **Eficiência de Memória:** Devem ser utilizadas estruturas uint8 e aproximação simples de contornos para minimizar o footprint de RAM.
 - **Robustez:** O sistema deve manter a precisão da segmentação sob variações moderadas de iluminação através de limiarização automática de Otsu.

6. Planeamento e Estruturação Final

A implementação seguirá um cronograma técnico focado na validação contínua de cada etapa do pipeline:

1. **Ambiente e Ingestão:** Configuração do ecossistema Python/OpenCV e teste de leitura de sensores (BGR alignment).
2. **Limpeza e Suavização:** Implementação de filtros (ênfase no Median Blur para ruído "sal e pimenta") e ajuste de kernels.
3. **Segmentação Automática:** Aplicação do Threshold de Otsu para definição dinâmica do limiar de binarização.
4. **Mapeamento Topológico:** Implementação do findContours com RETR_TREE para captura de relações hierárquicas.
5. **Vetorialização Compacta:** Aplicação de chain_approx_simple para otimização de armazenamento de contornos.
6. **Interpretação Semântica:** Integração com algoritmos de agrupamento (K-Means) ou Redes Convolutivas (CNN) para classificação de atributos e análise de conteúdo.